# Waste & Recycling Management

Advanced Web project – Design documents


1. Use case diagram (overview) p. 2

2. Use case – Plan a collection (Manager) p. 3

3. Use case – Carry out a collection round (Driver) p. 4

4. Database – Table diagram (physical model) p. 5

5. Database – Conceptual view p. 6

6. Planned Gantt chart p. 7

7. Member diagrams – Amadeo Berthelot (wireframe, activity, sequence) p. 8–10

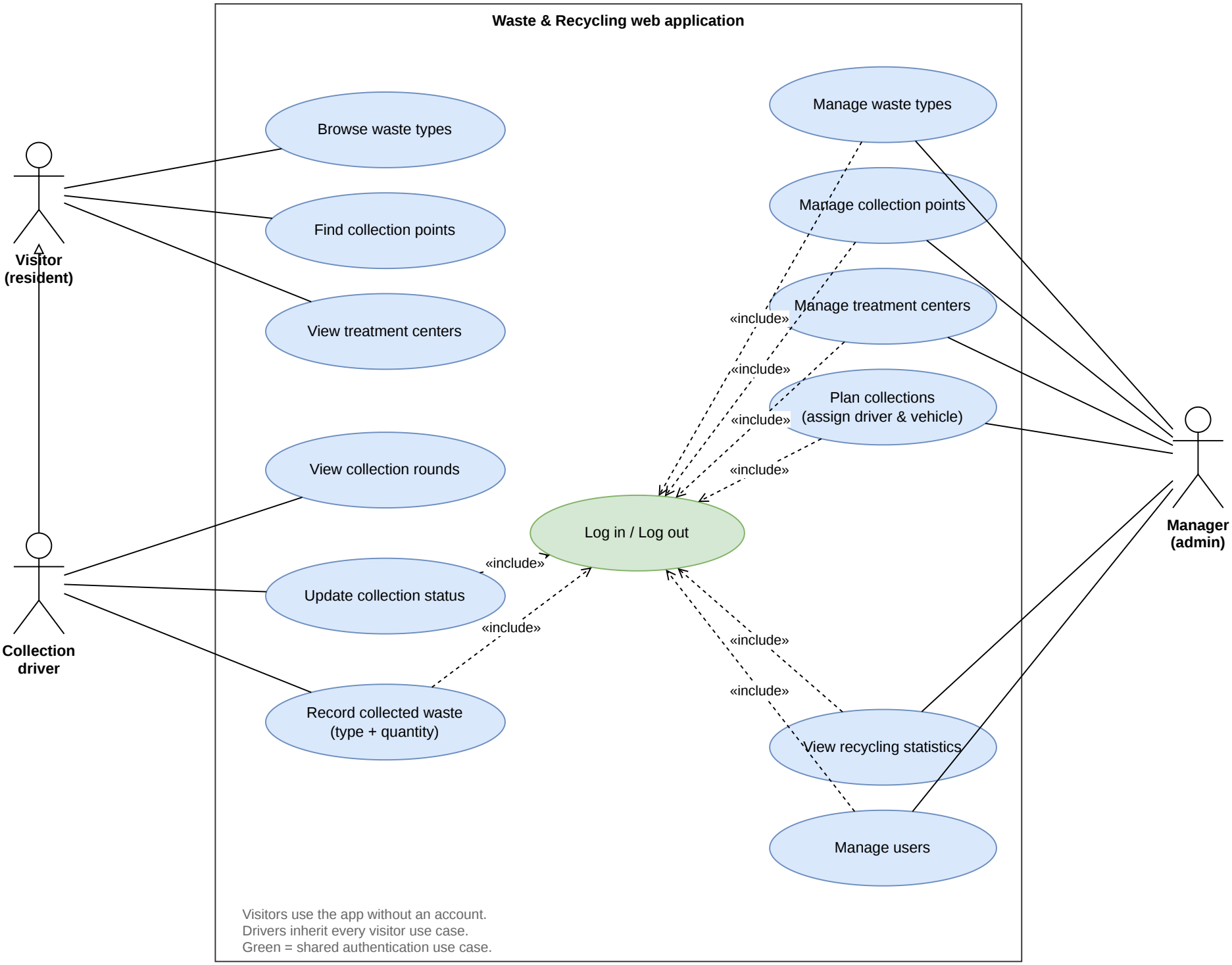
8. Member diagrams – Jules Venot (wireframe, activity, sequence) p. 11–13

9. Member diagrams – Hadi Daccache (wireframe, activity, sequence) p. 14–16



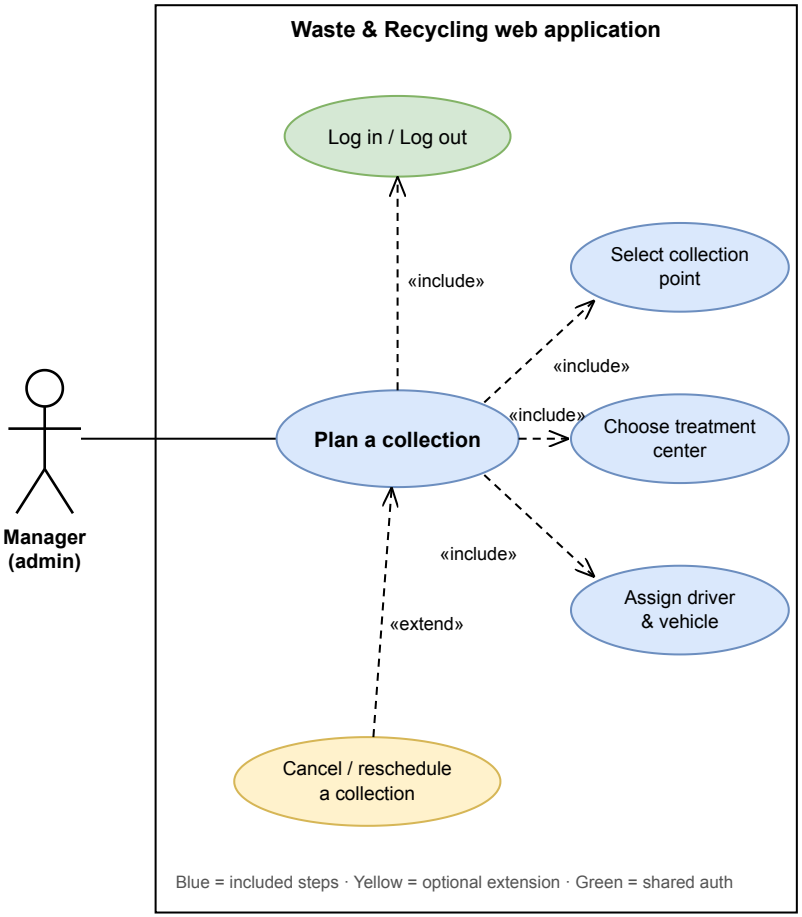
Jules Venot · Amadeo Berthelot · Hadi Daccache

# Waste & Recycling Management – Use Case Diagram



# Use Case – Plan a collection

Actor: Manager (admin) · Tables: collection, collection_point, treatment_center, app_user




| | |
|---|---|
| **ID / Name** | UC-A – Plan a collection |
| **Primary actor** | Manager (role *manager* in *app_user*) |
| **Goal** | Schedule the pickup of a collection point on a given date, linked to a treatment center, with a driver and a vehicle. |
| **Preconditions** | The manager is logged in. At least one collection point and one **active** treatment center exist. |
| **Postconditions** | A new row is stored in *collection* with status **planned**; it appears in the drivers' "collection rounds" list. |
| **Trigger** | A bin is full, or the weekly planning of rounds is prepared. |

## Main success scenario

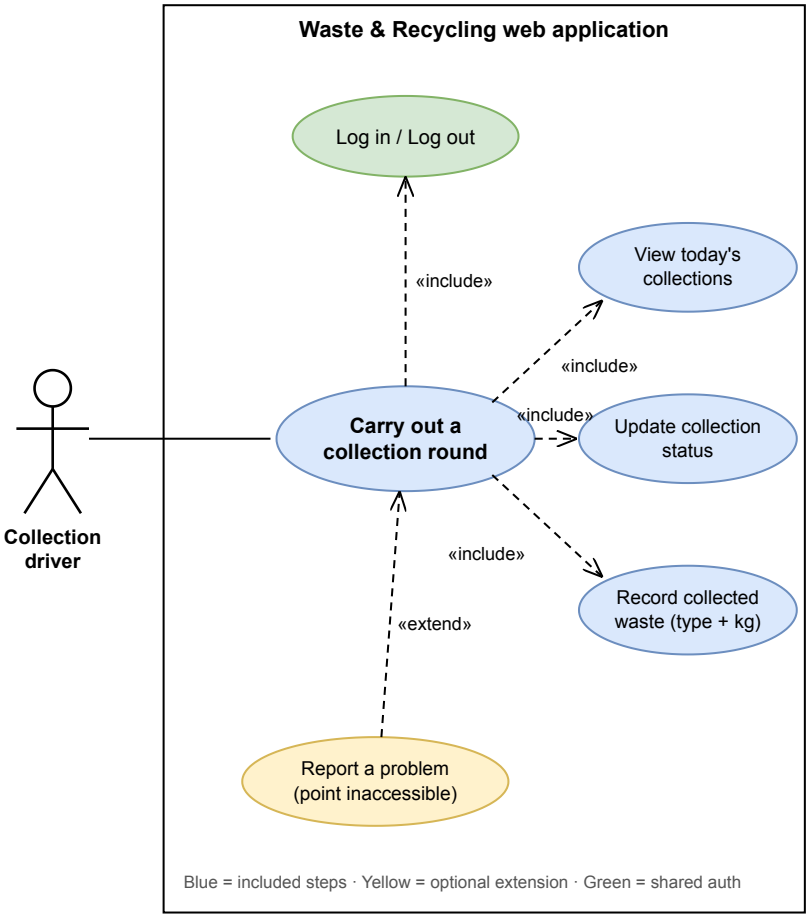
1. The manager opens the **Collections** page and clicks "Plan a collection".
2. The system displays a form with the list of collection points.
3. The manager selects a collection point.
4. The system pre-selects the treatment center linked to this point (*collection_point.center_id*).
5. The manager chooses the collection date, a driver and a vehicle plate.
6. The manager confirms the form.
7. The system validates the data, saves the collection with status **planned** and shows it in the list.

## Alternative / error flows

- **4a.** The point has no linked center (deleted center → NULL): the manager picks an active center manually.
- **5a.** The date is in the past: the system refuses and asks for a valid date.
- **5b.** The driver already has a collection at that date: the system shows a warning; the manager confirms or picks another driver.
- **7a.** The selected center is inactive (*is_active* = false): the collection is rejected.
- **Extension.** Later, the manager can reschedule a planned collection or set it to **cancelled**.

# Use Case – Carry out a collection round

Actor: Collection driver · Tables: collection, collection_waste, waste_type



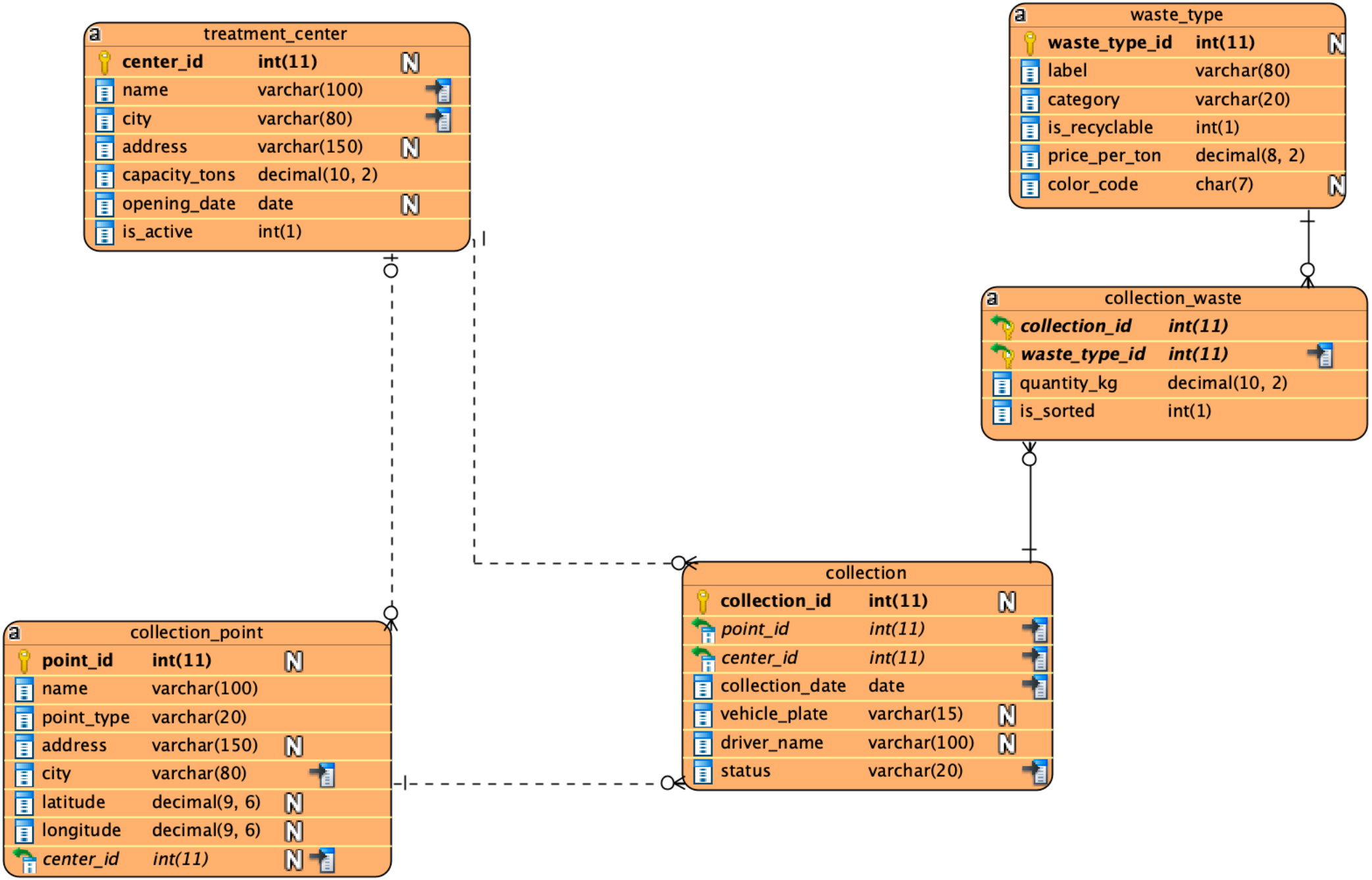

| | |
|---|---|
| **ID / Name** | UC-B – Carry out a collection round |
| **Primary actor** | Collection driver (role *driver* in *app_user*) |
| **Goal** | Follow the collections planned for the day, update their status and record the quantity of each type of waste collected. |
| **Preconditions** | The driver is logged in. At least one collection with status **planned** is assigned to the driver for today. |
| **Postconditions** | The collection is **completed** and one row per waste type is stored in *collection_waste*; the recycling statistics are updated. |
| **Trigger** | The driver starts the daily round. |

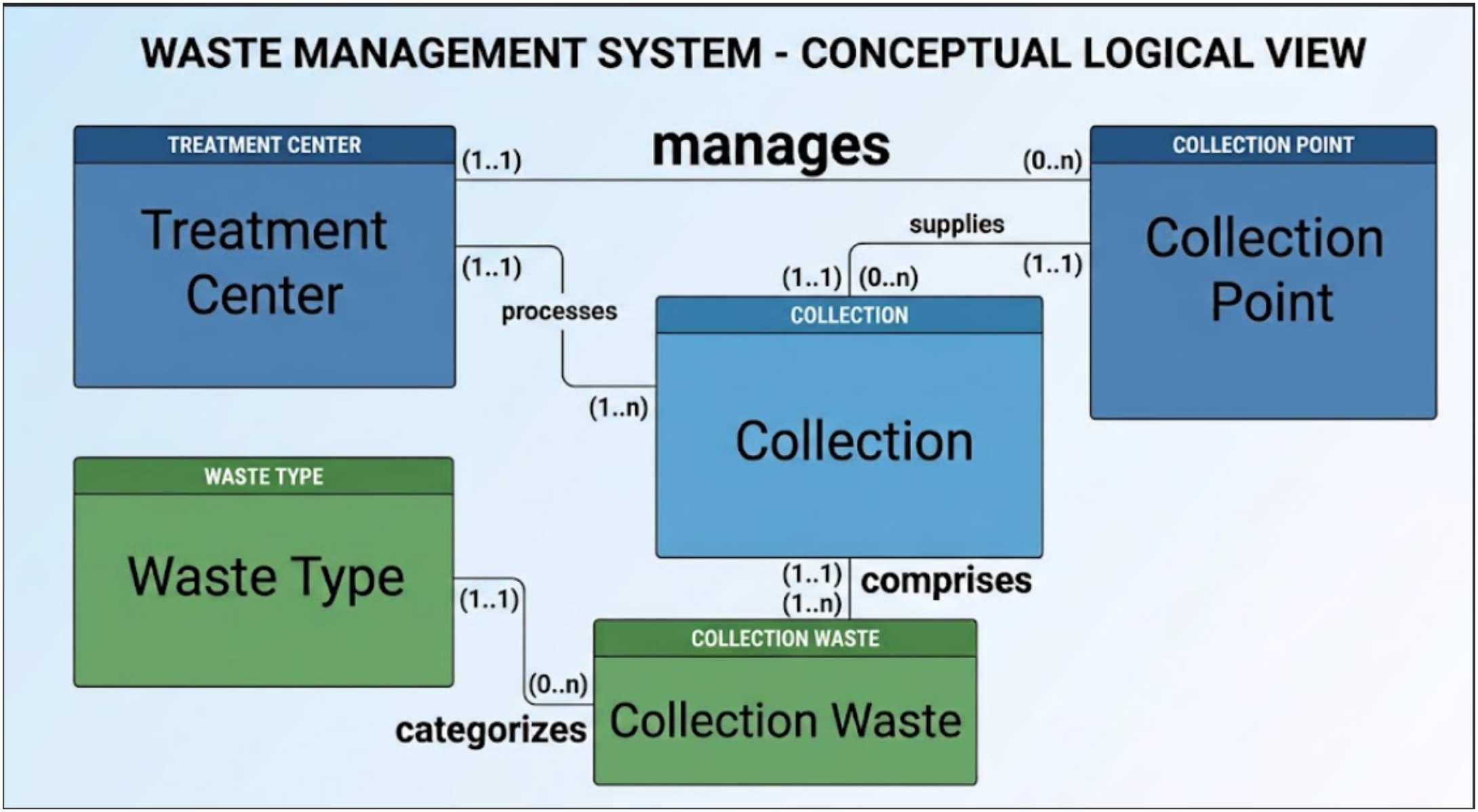
## Main success scenario

1. The driver opens the **Collections** page; the system lists the collections assigned for today.
2. On arrival at a collection point, the driver sets the collection status to **in_progress**.
3. The driver adds a line for each waste type collected: waste type, quantity (kg) and “sorted” yes/no.
4. The system checks the values and saves them in *collection_waste*.
5. The driver sets the status to **completed**.
6. The system confirms and the collection disappears from the list of remaining tasks.
7. Steps 2–6 are repeated for every point of the round.

## Alternative / error flows

- **3a.** The same waste type is entered twice: the system refuses (primary key *collection_id* + *waste_type_id*) and offers to edit the existing line.
- **3b.** Quantity is negative or empty: the system shows an error.
- **5a.** No waste recorded: the system asks for confirmation before setting the status to completed (empty bin).
- **Extension.** The point is inaccessible (road works, blocked bin): the driver reports the problem and the collection is set to **cancelled**; the manager is notified to reschedule it.


treatment_center
center_id int(11) N
name varchar(100)
city varchar(80)
address varchar(150) N
capacity_tons decimal(10, 2)
opening_date date N
is_active int(1)
waste_type
waste_type_id int(11) N
label varchar(80)
category varchar(20)
is_recyclable int(1)
price_per_ton decimal(8, 2)
color_code char(7) N
collection_waste
collection_id int(11)
waste_type_id int(11)
quantity_kg decimal(10, 2)
is_sorted int(1)
collection
collection_id int(11) N
point_id int(11)
center_id int(11)
collection_date date
vehicle_plate varchar(15) N
driver_name varchar(100) N
status varchar(20)
collection_point
point_id int(11) N
name varchar(100)
point_type varchar(20)
address varchar(150) N
city varchar(80)
latitude decimal(9, 6) N
longitude decimal(9, 6) N
center_id int(11) N


WASTE MANAGEMENT SYSTEM - CONCEPTUAL LOGICAL VIEW
TREATMENT CENTER
Treatment Center
(1..1)
manages
(0..n)
COLLECTION POINT
Collection Point
supplies
(1..1)
(0..n)
(1..1)
processes
(1..n)
COLLECTION
Collection
WASTE TYPE
Waste Type
(1..1)
(1..1)
(1..n)
comprises
(0..n)
categorizes
COLLECTION WASTE
Collection Waste

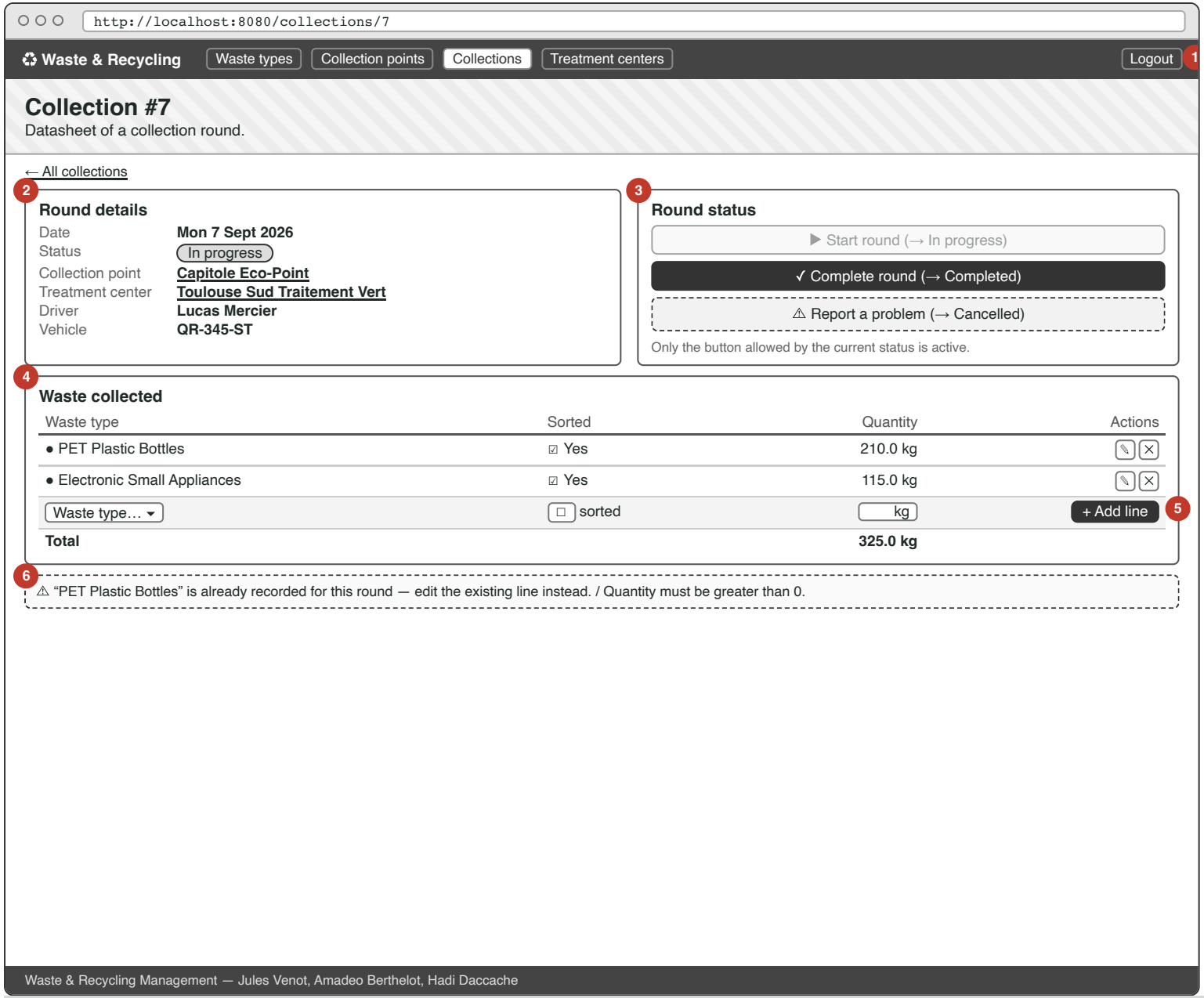
# Waste & Recycling Management – Planned Gantt (3 Sep → 17 Dec 2026)

| Task | Owner | W1 31 Aug | W2 07 Sep | W3 14 Sep | W4 21 Sep | W5 28 Sep | W6 05 Oct | W7 12 Oct | W8 19 Oct | W9 26 Oct | W10 02 Nov | W11 09 Nov | W12 16 Nov | W13 23 Nov | W14 30 Nov | W15 07 Dec | W16 14 Dec |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **PHASE 1 – Analysis & database** | All | | | | | | | | | | | | | | | | |
| Topic, team, entities, planned functions (CRUD + non-CRUD) | All | | | | | | | | | | | | | | | | |
| ER diagram + table structure diagram (PDF) | Jules | | | | | | | | | | | | | | | | |
| ◆ 10 Sep – email: topic + ER / table diagrams | Jules | | | | | | | | | | | | | | | | |
| Full SQL (drop/create/insert) + fake JSON (10+ rows) | Hadi | | | | | | | | | | | | | | | | |
| Git repo, .gitignore, README | Amadeo | | | | | | | | | | | | | | | | |
| ◆ 17 Sep – SQL + JSON + diagrams PDF in git | Amadeo | | | | | | | | | | | | | | | | |
| **PHASE 2 – Frontend (Vue.js, fake JSON)** | All | | | | | | | | | | | | | | | | |
| Vue setup, router, navbar, layout | Amadeo | | | | | | | | | | | | | | | | |
| List / table pages (4 tables) | Amadeo / Hadi | | | | | | | | | | | | | | | | |
| Use case diagram + planned Gantt | All | | | | | | | | | | | | | | | | |
| ◆ 15 Oct – use case, Gantt, 3 list pages, branches | All | | | | | | | | | | | | | | | | |
| Wireframe, activity, sequence (1 each per member) | All | | | | | | | | | | | | | | | | |
| Datasheet + editor pages: full CRUD on JSON | Hadi / Amadeo | | | | | | | | | | | | | | | | |
| ◆ 29 Oct – full frontend + member diagrams | Hadi / Amadeo | | | | | | | | | | | | | | | | |
| ◆ 4 Nov – Midterm | All | | | | | | | | | | | | | | | | |
| **PHASE 3 – Server (Express + MySQL)** | All | | | | | | | | | | | | | | | | |
| Express setup, MVC pipeline, DB controllers | Jules | | | | | | | | | | | | | | | | |
| Component (per member) + class diagram | All | | | | | | | | | | | | | | | | |
| ◆ 26 Nov – server in git + component / class diagrams | Jules | | | | | | | | | | | | | | | | |
| List controllers + Postman tests | Jules / Hadi | | | | | | | | | | | | | | | | |
| Frontend fetch: at least 3 server reads | Amadeo | | | | | | | | | | | | | | | | |
| Update diagrams to the real implementation + TODO list | All | | | | | | | | | | | | | | | | |
| ◆ 4 Dec – big project recheck | All | | | | | | | | | | | | | | | | |
| **PHASE 4 – Finish & exam** | All | | | | | | | | | | | | | | | | |
| Authentication (bcrypt) + authorization | Jules | | | | | | | | | | | | | | | | |
| Full CRUD via server + business logic (views, GROUP BY, JOIN stats) | Hadi / Amadeo | | | | | | | | | | | | | | | | |
| All diagrams in one PDF + actual Gantt | All | | | | | | | | | | | | | | | | |
| ◆ 17 Dec – project exam | All | | | | | | | | | | | | | | | | |

Project mgmt / UML  Database  Frontend  Backend  Quality & docs

# Wireframe — Collection datasheet (driver view)

Screen used during use case **UC-B "Carry out a collection round"** · route `/collections/:id` · tables `collection`, `collection_waste`, `waste_type`



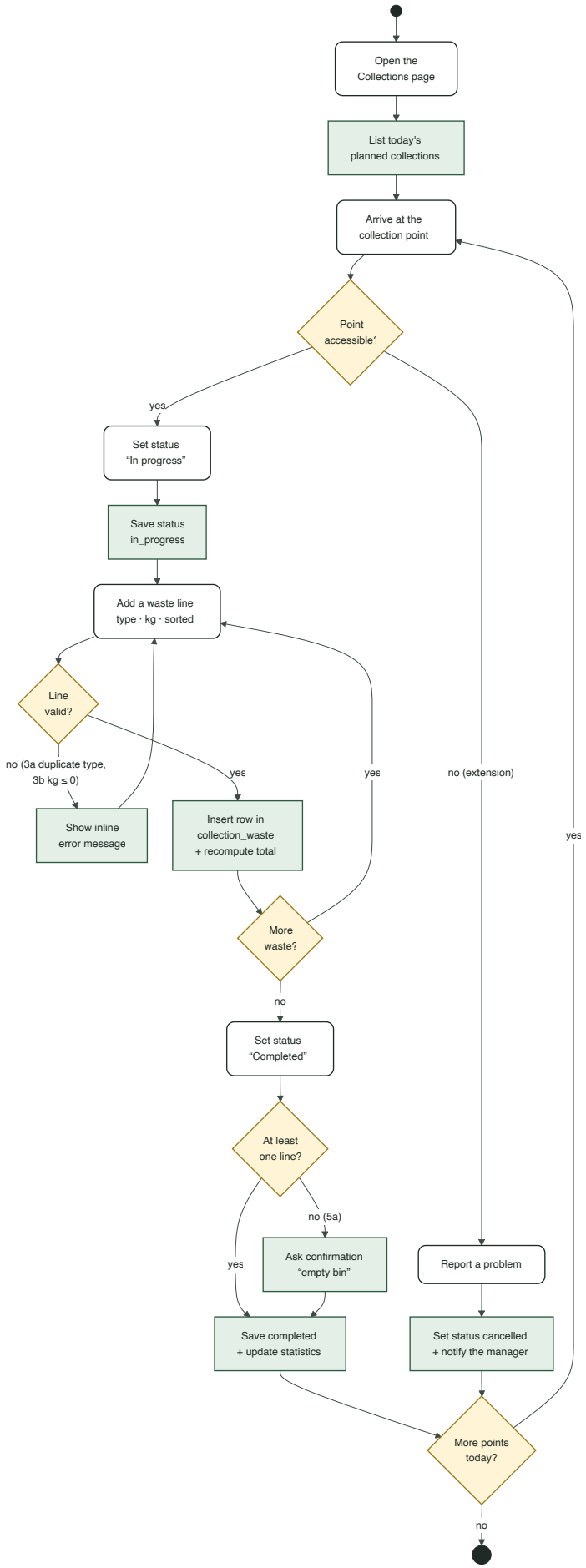

## Annotations

1. Shared navbar; the current section (*Collections*) is highlighted.
2. Read-only facts of the `collection` row. Point and center are links to their own datasheets (FK `point_id`, `center_id`).
3. Status buttons follow the ENUM: *planned* → *in_progress* → *completed*, or *cancelled* when the point is inaccessible (UC-B steps 2, 5 and extension).
4. One row per `collection_waste` line (JOIN with `waste_type` for the label and colour). Each line can be edited or removed.
5. Inline "add line" row: waste type, sorted yes/no, quantity in kg. The total is recalculated after each change (SUM of `quantity_kg`).
6. Inline error message, no pop-up: duplicate waste type (PK `collection_id + waste_type_id`, flow 3a) or empty / negative quantity (flow 3b).

**Legend**
Dark button = main action · dashed border = exceptional action / message · ▾ = drop-down · □ = checkbox · hatched area = header picture.

# Activity diagram — Carry out a collection round

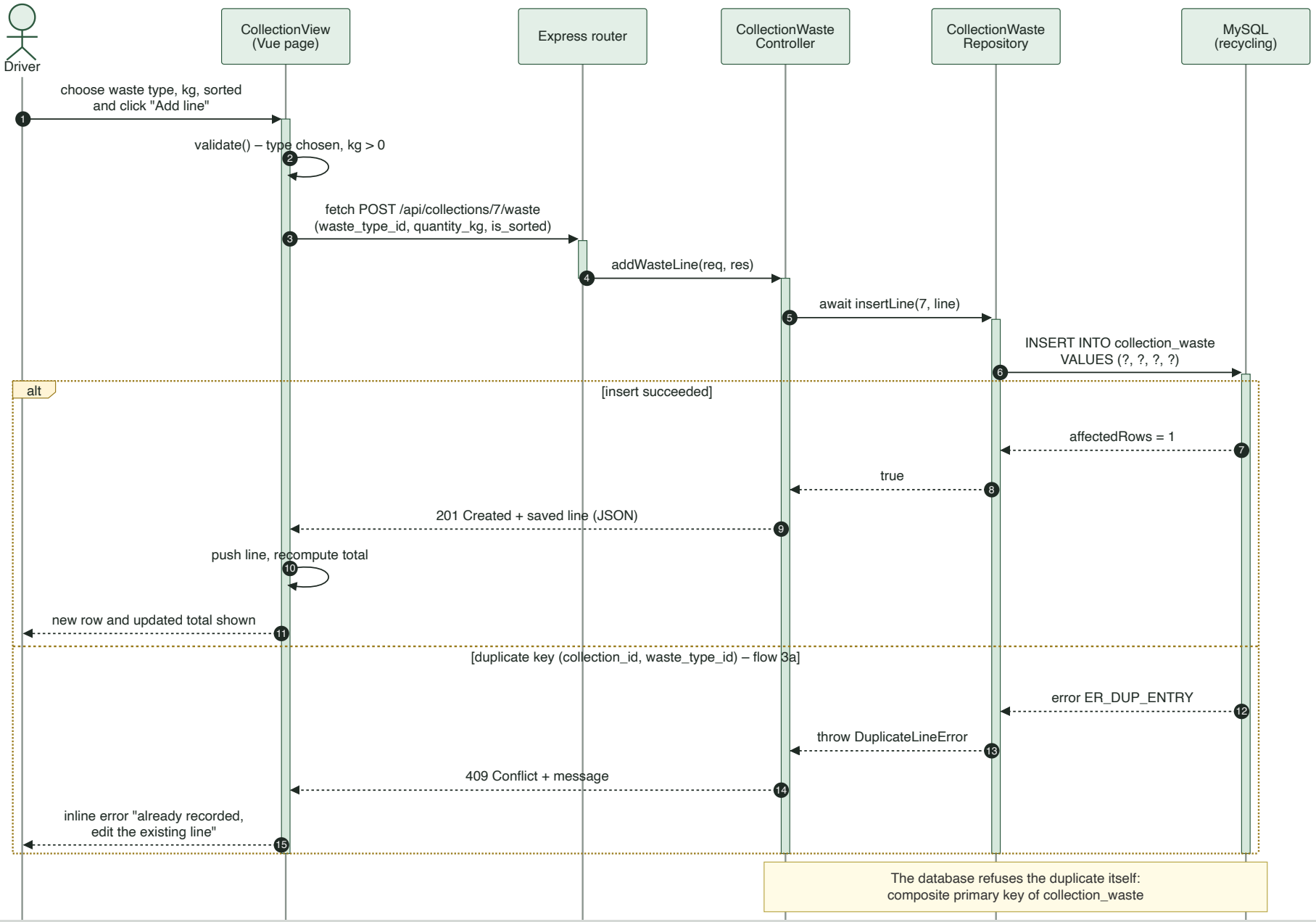
Use case **UC-B** · actor: collection driver (role `driver` in `app_user`) · includes alternative flows 3a, 3b, 5a and the “point inaccessible” extension




Source: docs/diagrams/src/amadeo_activity.mmd (Mermaid, editable on mermaid.live)

# Sequence diagram — Record a collected waste line

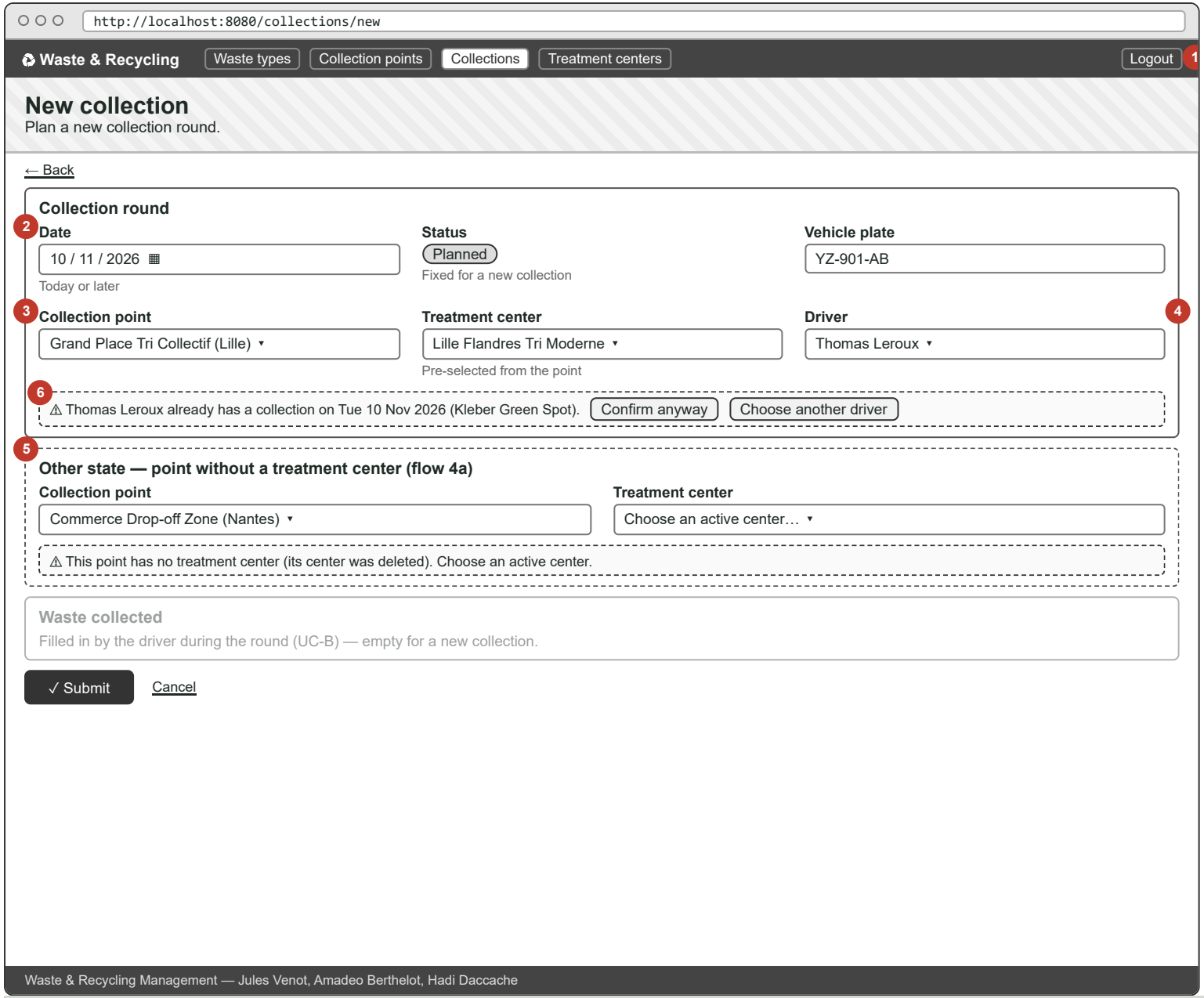
UC-B steps 3–4 · planned server flow (Express MVC pipeline + repository) · `POST /api/collections/:id/waste` → `INSERT INTO collection_waste`




Source: docs/diagrams/src/amadeo_sequence.mmd (Mermaid, editable on mermaid.live) · solid arrow = call, dashed = return

# Wireframe — New collection form (manager view)

Screen used during use case **UC-A "Plan a collection"** · route `/collections/new` · tables `collection`, `collection_point`, `treatment_center`, `app_user`



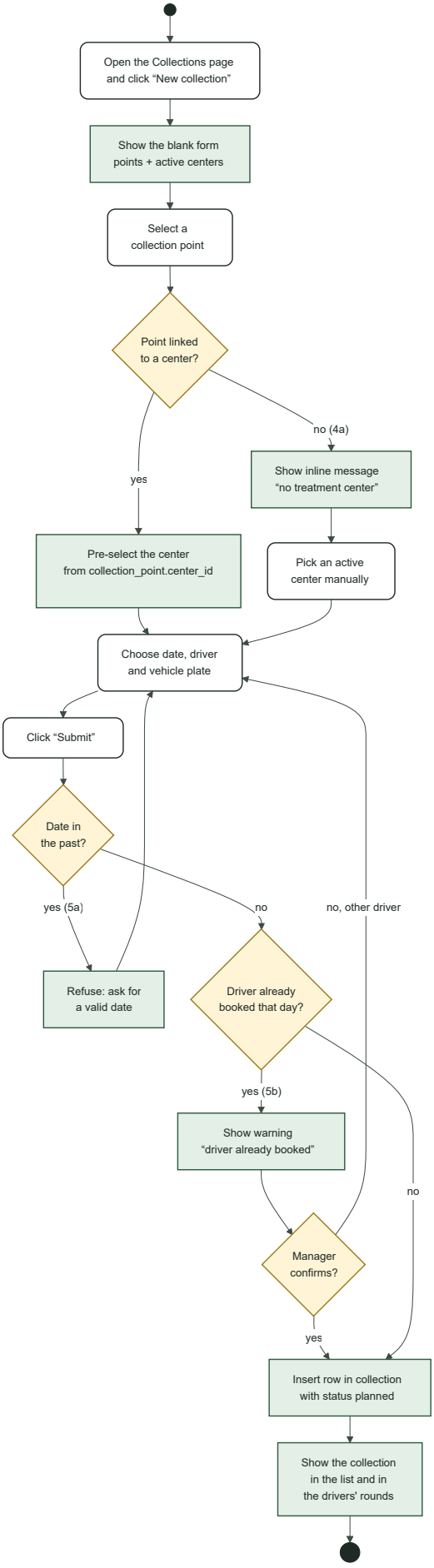

## Annotations

1. Shared navbar; the current section (*Collections*) is highlighted. The "New collection" button on top of the list opens this blank form.
2. Date picker: the date must be today or later, otherwise the form is refused (flow 5a). The status is fixed to *planned* for a new collection (ENUM default).
3. Drop-down filled from `collection_point` (name + city). Choosing a point pre-selects its center from `collection_point.center_id` (step 4). Only active centers (`is_active`) are listed.
4. Driver chosen among the users with role `driver` in `app_user`; stored with the plate in `collection.driver_name` and `vehicle_plate`.
5. Flow 4a: the point has no center (FK `ON DELETE SET NULL`) → inline error, the manager picks an active center manually.
6. Flow 5b: the driver already has a collection that day → warning, not a blocking error. Submit sends `POST /api/collections`; Cancel goes back without saving.

**Legend**
Dark button = main action · dashed border = exceptional action / message · ▾ = drop-down · ▦ = date picker · faded box = not used in this screen · hatched area = header picture.

# Activity diagram — Plan a collection

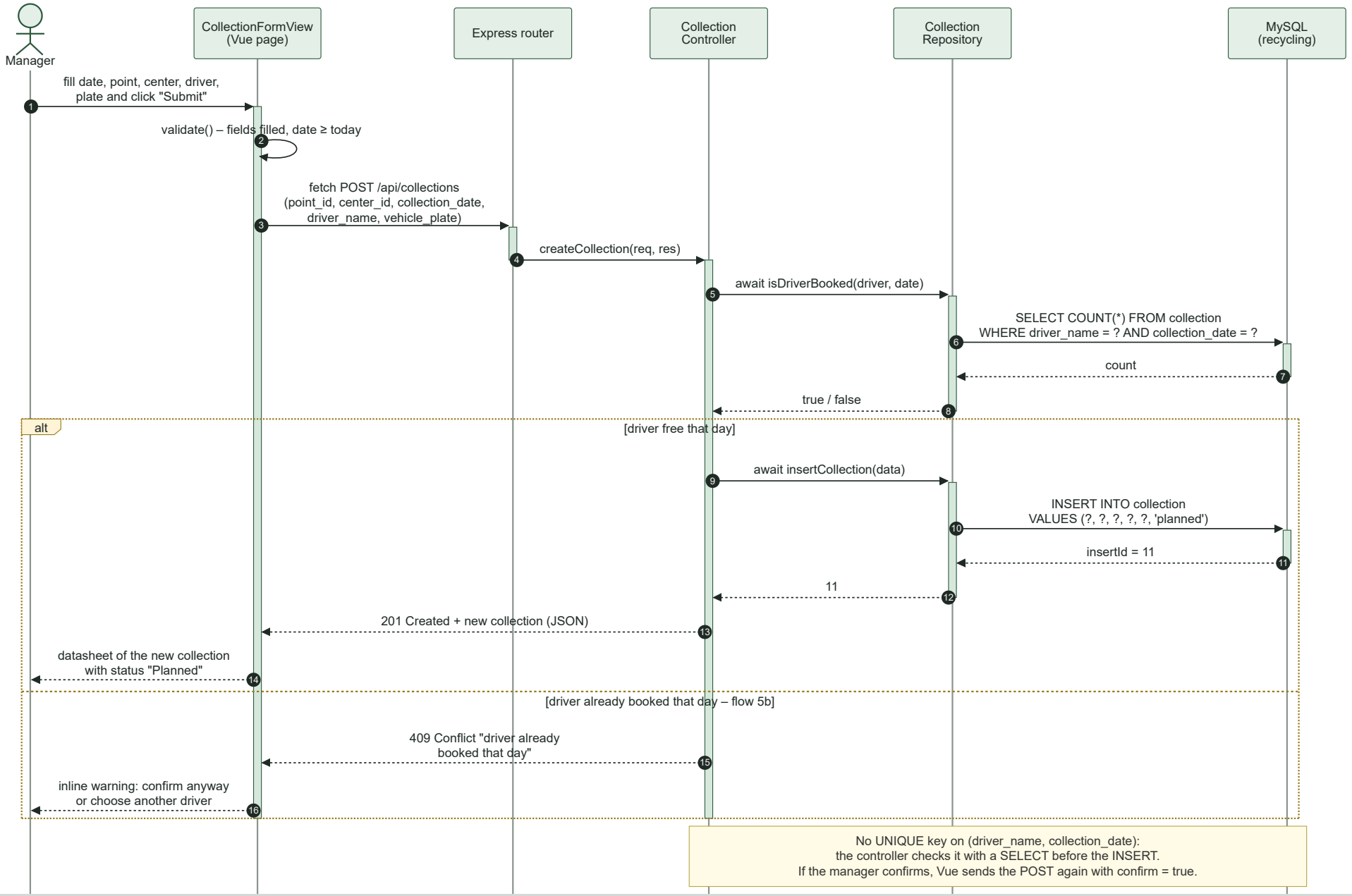
Use case **UC-A** · actor: manager (role `manager` in `app_user`) · alternative flows 4a, 5a and 5b




manager action · system action (Vue + server) · decision · ● start · ◉ end · numbers = UC-A alternative flows

# Sequence diagram — Save a new collection

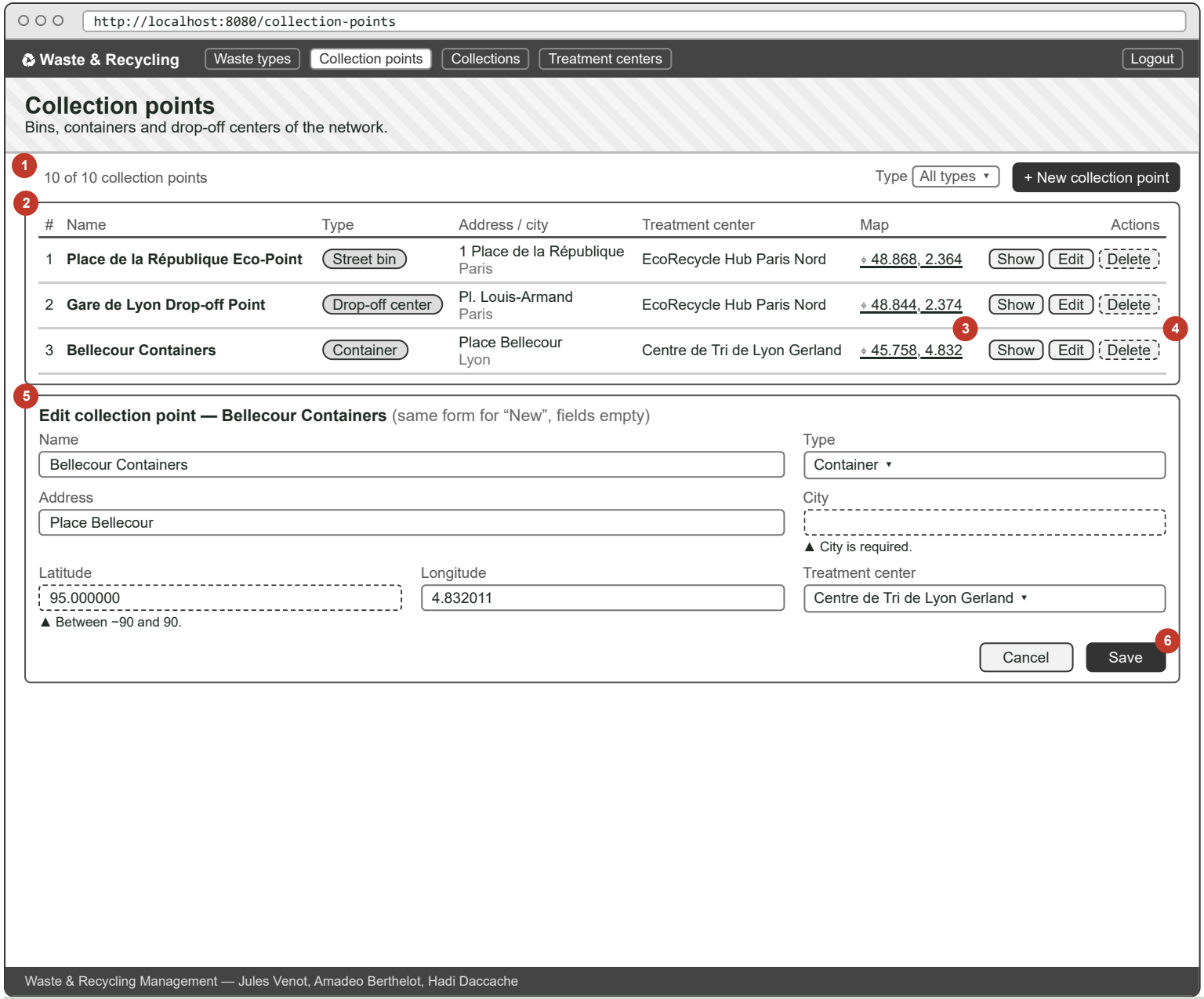
UC-A steps 6–7 · planned server flow (Express MVC pipeline + repository) · `POST /api/collections` → `INSERT INTO collection`




Source: docs/diagrams/src/jules_sequence.mmd (Mermaid, editable on mermaid.live) · solid arrow = call, dashed = return

# Wireframe — Collection points (manager view)

Screens used to manage the network: list `/collection-points` and form `/collection-points/new` · `/collection-points/:id/edit` · tables `collection_point`, `treatment_center`



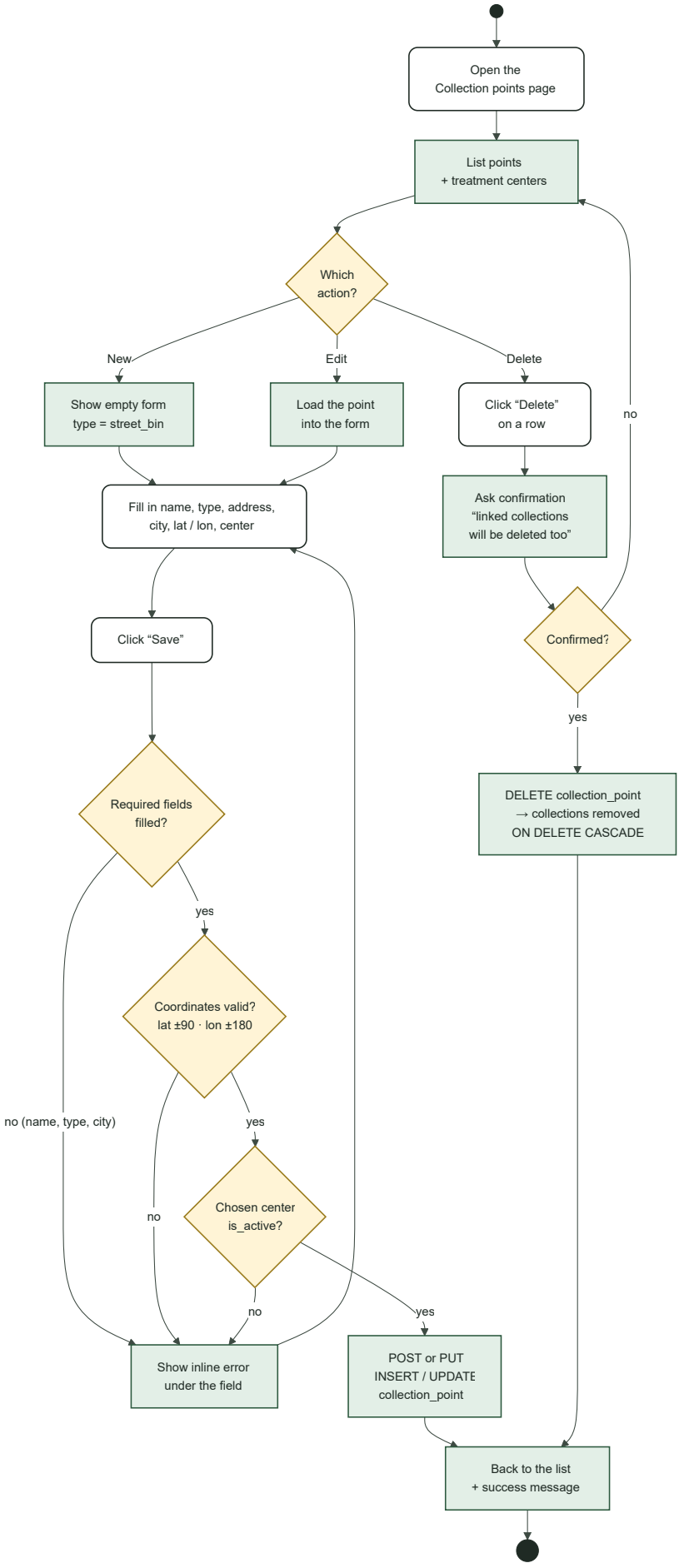

## Annotations

1. Toolbar: counter, filter on the `point_type` ENUM (*street_bin, drop_off_center, container, business*) and the main action “New collection point”.
2. One row per `collection_point`. The center name comes from the FK `center_id` (JOIN with `treatment_center`), “—” if none.
3. Map link built from `latitude` / `longitude`, opens OpenStreetMap in a new tab.
4. Shared Show / Edit / Delete buttons. Delete asks for confirmation: the linked collections are deleted too (`ON DELETE CASCADE`).
5. Form with the table columns: name, type, address, city, latitude, longitude, treatment center (drop-down of **active** centers only).
6. “Save” checks the fields first; errors are shown inline under each field, no pop-up. On success → POST / PUT, then back to the list.

**Legend**
Dark button = main action · dashed border = destructive action / invalid field · ▾ = drop-down · ⌖ = map link · hatched area = header picture.

# Activity diagram — Add or edit a collection point

Network management · actor: manager (role `manager` in `app_user`) · create / edit with validations, plus deletion with confirmation




manager action · system action (Vue + server) · decision · ● start · ◉ end · any validation error loops back to the form

Source: docs/diagrams/src/hadi_activity.mmd (Mermaid, editable on mermaid.live)

# Sequence diagram — Edit a treatment center

Planned server flow (Express MVC pipeline + repository) · `PUT /api/treatment-centers/:id` → `UPDATE treatment_center … WHERE center_id = ?`

